

**SVEUČILIŠTE U ZAGREBU
FAKULTET ORGANIZACIJE I INFORMATIKE
VARAŽDIN**

Luka Kukec

**GENERIRANJE RAZINA IGRE
KORIŠTENJEM GRAMATIKE GRAFOVA**

PROJEKT

BAZE PODATAKA I BAZE ZNANJA

Varaždin, 2026.

SVEUČILIŠTE U ZAGREBU
FAKULTET ORGANIZACIJE I INFORMATIKE
V A R A Ž D I N

Luka Kukec

Matični broj: 0016158557

Studij: Baze podataka i baze znanja

GENERIRANJE RAZINA IGRE KORIŠTENJEM GRAMATIKE
GRAFOVA

PROJEKT

Mentor:

doc. dr. sc. Bogdan Okreša Đurić

Varaždin, siječanj 2026.

Luka Kukec

Izjava o izvornosti

Izjavljujem da je ovaj projekt izvorni rezultat mojeg rada te da se u izradi istoga nisam koristio drugim izvorima osim onima koji su u njemu navedeni. Za izradu rada su korištene etički prikladne i prihvatljive metode i tehnike rada.

Autor potvrdio prihvatanjem odredbi u sustavu FOI Radovi

Sažetak

Ovaj rad opisuje implementaciju sustava za proceduralno generiranje razina u računalnim igrama koristeći gramatike grafova (engl. Graph Grammars). Sustav je implementiran u programskom jeziku Prolog s vizualizacijom u Pythonu (PyGame). Rad proučava teorijsku podlogu gramatika grafova i njihovu primjenu u stvaranju kompleksnih i varijabilnih struktura tamnica (dungeons).

Ključne riječi: gramatike grafova; proceduralno generiranje sadržaja; Prolog; Python; razvoj igara; roguelike

Sadržaj

1. Uvod	1
2. Teorijska podloga	2
2.1. Proceduralno generiranje sadržaja	2
2.2. Gramatike grafova	2
2.3. L-sustavi	2
2.4. Logičko programiranje i Prolog	2
2.5. Formalizacija sustava	3
2.5.1. Definicija predikata	3
2.5.2. Formalna pravila	3
3. Opis implementacije	5
3.1. Arhitektura sustava	5
3.2. Generator u Prologu	5
3.2.1. Definicija gramatike	5
3.2.2. Probabilistička gramatika	6
3.2.3. Algoritam selekcije	6
3.2.4. Algoritam ekspanzije	6
3.3. Integracija i Vizualizacija	7
4. Kritički osvrt	9
4.1. Prednosti	9
4.2. Nedostaci	9
4.3. Mogući nastavak	9
5. Prikaz rada aplikacije	10
6. Zaključak	11
Popis literature	12
Popis slika	13

1. Uvod

Računalne igre su kompleksni softverski sustavi koji zahtijevaju velike količine sadržaja kako bi održali interes igrača. Jedan od ključnih elemenata mnogih igara, posebno onih u žanru *roguelike*, je dizajn razina. Ručno dizajniranje razina je vremenski zahtjevan proces koji ograničava količinu sadržaja koju razvojni tim može proizvesti. Kako bi se riješio ovaj problem, koristi se proceduralno generiranje sadržaja (engl. *Procedural Content Generation - PCG*).

Ovaj rad bavi se proceduralnim generiranjem razina u igri korištenjem gramatika grafova (engl. *Graph Grammars*). Motivacija za ovaj pristup leži u mogućnosti definiranja strukture razine putem skupa logičkih pravila, što omogućuje generiranje topološki ispravnih i zanimljivih tlocrta tamnica. Za razliku od jednostavnih nasumičnih algoritama, gramatike grafova nude bolju kontrolu nad strukturom i povezanošću prostorija [1].

Cilj rada je implementirati sustav koji koristi deklarativnu paradigmu (Prolog) za generiranje strukture razine na temelju gramatike, te imperativnu/objektno-orijentiranu paradigmu (Python) za vizualizaciju i interakciju s igračem.

2. Teorijska podloga

2.1. Proceduralno generiranje sadržaja

Proceduralno generiranje sadržaja (PCG) odnosi se na stvaranje sadržaja igre automatski, putem algoritama, s ograničenim ili neizravnim unosom korisnika [2]. PCG se koristi za generiranje različitih vrsta sadržaja, uključujući tekstore, modele, glazbu, priču i razine [3]. U kontekstu ovog rada, fokus je na generiranju razina, odnosno tlocrta prostora (tamnica).

Postoje različiti pristupi PCG-u, od jednostavnih nasumičnih algoritama do sofisticiranih metoda poput gramatika grafova i L-sustava. Sustav Tanagra [4] demonstrira kako se reaktivno planiranje može koristiti za interaktivni dizajn razina.

2.2. Gramatike grafova

Gramatike grafova su formalizam za specificiranje klasa grafova i operacija nad njima. Slično kao što Chomskyjeve gramatike definiraju pravila za generiranje nizova znakova (simbola), gramatike grafova definiraju pravila za generiranje grafova [1].

Gramatika grafa sastoji se od početnog grafa (aksioma) i skupa produkcijskih pravila (pravila prepisivanja). Svako pravilo definira podgraf koji se traži u trenutnom grafu (lijeva strana pravila) i podgraf kojim se on zamjenjuje (desna strana pravila).

Primjena gramatika grafova u generiranju razina omogućuje dizajnerima da definiraju "recepte" za strukturu razine [5]. Na primjer, pravilo može reći: "Ako postoji soba s blagom, dodaj sobu s čudovištem ispred nje". Ovo osigurava da generirane razine imaju smisla u kontekstu igrivosti (engl. *gameplay*).

2.3. L-sustavi

L-sustavi (Lindenmayerovi sustavi) su paralelni sustavi za prepisivanje koji su originalno razvijeni za modeliranje rasta biljaka [6]. Sastoje se od abecede simbola, aksioma (početnog niza) i skupa produkcijskih pravila. L-sustavi su srodne s gramatikama grafova i koriste se u proceduralnom generiranju za stvaranje fraktalnih i organskih struktura [7].

U kontekstu generiranja razina, L-sustavi mogu poslužiti za generiranje hodnika i povezivanje prostorija, dok gramatike grafova definiraju topološku strukturu tamnice.

2.4. Logičko programiranje i Prolog

Prolog (Programming in Logic) je deklarativni programski jezik koji se temelji na predikatnoj logici prvog reda [8]. Program u Prologu sastoji se od činjenica i pravila. Prolog je izuzetno pogodan za rješavanje problema koji uključuju simboličku manipulaciju, pretraživanje

prostora stanja i zaključivanje, što ga čini idealnim alatom za implementaciju gramatika grafova.

U ovom radu Prolog se koristi za definiranje pravila gramatike i izvođenje procesa generiranja pretraživanjem i zamjenom simbola (čvorova grafa).

2.5. Formalizacija sustava

Kako bi se osigurala ispravnost i konzistentnost generiranih razina, sustav se može opisati formalnim jezikom predikatnog računa.

2.5.1. Definicija predikata

Osnovni građevni blokovi sustava definirani su sljedećim predikatima:

- $room(id, x, y) : \mathbb{N} \times \mathbb{Z} \times \mathbb{Z}$ – Soba s jedinstvenim identifikatorom id na koordinatama (x, y) .
- $connected(a, b, dir) : \mathbb{N} \times \mathbb{N} \times Dir$ – Soba a povezana je sa sobom b u smjeru dir , gdje je $Dir = \{north, south, east, west\}$.
- $room_type(id, type) : \mathbb{N} \times Type$ – Soba id je tipa $type$, gdje je $Type = \{start, boss, combat, treasure, \dots\}$.

2.5.2. Formalna pravila

Sustav mora zadovoljiti skup logičkih ograničenja (constraints):

Pravilo 1: Jedinstvenost početne i završne sobe Postoji točno jedna soba tipa *start* i točno jedna soba tipa *boss*.

$$\exists! s \in Rooms : room_type(s, start) \wedge \exists! b \in Rooms : room_type(b, boss) \quad (2.1)$$

Pravilo 2: Povezanost grafa Za svake dvije sobe r_1, r_2 mora postojati put između njih.

$$\forall r_1, r_2 \in Rooms : path(r_1, r_2) \quad (2.2)$$

Pravilo 3: Minimalna udaljenost Udaljenost između početne i završne sobe mora biti značajna (veća od ili jednaka 2).

$$\forall s, b : (room_type(s, start) \wedge room_type(b, boss)) \implies distance(s, b) \geq 2 \quad (2.3)$$

Pravilo 4: Topologija soba s blagom Sobe s blagom su "slijepoj ulici" (dead-end), odnosno imaju samo jedan izlaz.

$$\forall r : room_type(r, treasure) \implies degree(r) = 1 \quad (2.4)$$

Ovaj formalni model omogućuje tretiranje problema generiranja razine kao Problema zadovoljenja ograničenja (Constraint Satisfaction Problem - CSP), kojeg Prolog rješava mehanizmom unifikacije i backtrackinga.

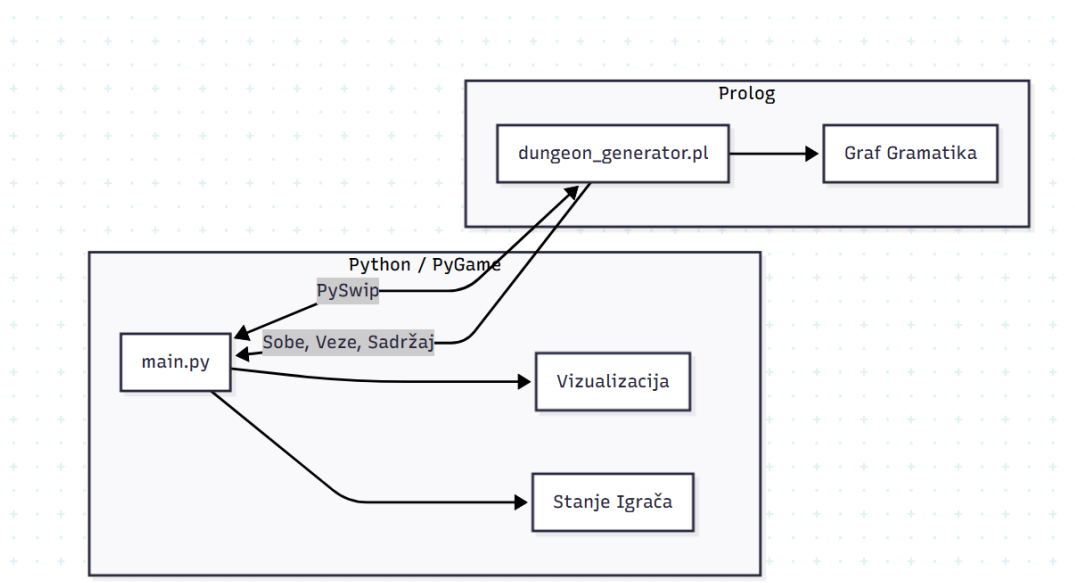
3. Opis implementacije

Sustav se sastoji od dviju glavnih komponenti: generatora razina implementiranog u Prologu i grafičkog sučelja te logike igre implementiranih u Pythonu. Ove dvije komponente komuniciraju putem biblioteke *pyswip* (ili subprocess poziva), gdje Python šalje zahtjev za generiranjem, a Prolog vraća strukturu razine.

3.1. Arhitektura sustava

Arhitektura je podijeljena na:

- **Backend (Prolog):** Odgovoran za logičko generiranje tlocrta tamnice, određivanje tipova soba i njihovih veza.
- **Frontend (Python/PyGame):** Odgovoran za vizualizaciju, upravljanje stanjem igre (kretanje igrača, borba) i interakciju s korisnikom.



Slika 1: Dijagram arhitekture sustava

3.2. Generator u Prologu

Jezgra generatora nalazi se u datoteci `dungeon_generator.pl`. Generiranje započinje definiranjem gramatike grafa.

3.2.1. Definicija gramatike

Gramatika je definirana nizom pravila prepisivanja oblika `weighted_rule(ParentType, ChildType, Weight)`. Težina (`weight`) određuje relativnu vjerojatnost odabira tog tipa sobe. Pri-

mjer pravila iz koda:

Isječak koda 1: Primjer gramatičkih pravila

```
1 %% weighted_rule (ParentType, ChildType, Weight)
2
3 weighted_rule(start, combat, 60).
4 weighted_rule(start, event, 40).
5
6 weighted_rule(combat, combat, 50).
7 weighted_rule(combat, treasure, 30).
8 weighted_rule(combat, shop, 10).
9 weighted_rule(combat, event, 10).
10
11 weighted_rule(event, treasure, 40).
12 weighted_rule(event, empty, 30).
13 weighted_rule(event, combat, 30).
14
15 weighted_rule(empty, combat, 70).
16 weighted_rule(empty, event, 30).
17
18 weighted_rule(shop, combat, 100).
19
20 weighted_rule(treasure, boss, 60).
21 weighted_rule(treasure, combat, 40).
22
23 weighted_rule(boss, _, 0).
```

Ova pravila, prikazana u ispisu iznad, određuju koje nove sobe (čvorove) možemo stvoriti iz postojeće sobe određenog tipa. Veća težina znači veću vjerojatnost odabira - npr. shop ima samo 10% šanse, dok combat ima 60%.

3.2.2. Probabilistička gramatika

Za razliku od jednostavnih determinističkih gramatika, sustav koristi probabilističku gramatiku s težinama (engl. *weighted rules*). Svako pravilo ima pridruženu težinu koja određuje vjerojatnost njegove primjene, kao što je prikazano u popisu iznad.

3.2.3. Algoritam selekcije

Odabir tipa sobe koristi *roulette wheel* algoritam selekcije. Predikat `select_by_weight/2` računa kumulativne težine i odabire tip proporcionalno njegovoj težini.

3.2.4. Algoritam ekspanzije

Proces generiranja koristi predikat `expand_dungeon/4` koji implementira pretraživanje u širinu (BFS) za širenje gramatike. Algoritam pokušava postaviti nove sobe na slobodne susjedne pozicije:

Isječak koda 2: Algoritam ekspanzije tamnice

```
1 %% expand_dungeon(Queue, Count, Max, Iterations)
2 expand_dungeon([room(ID, Type, X, Y) | RestQueue], Count, Max, Iterations) :-
3     NewIterations is Iterations + 1,
4
5     ( grammar_rule(Type, NextTypes)
6     -> ShuffledTypes = NextTypes
7     ; ShuffledTypes = []
8     ),
9
10    spawn_neighbors(ID, X, Y, ShuffledTypes, RestQueue, NewQueue,
11                    Count, NewCount, Max),
12    expand_dungeon(NewQueue, NewCount, Max, NewIterations).
13
14 %% spawn_neighbors(...) pokušava postaviti sobe u nasumičnim smjerovima
15 spawn_neighbors(ParentID, X, Y, [NextType|RestTypes],
16                 QueueIn, QueueOut, Count, FinalCount, Max) :-
17     get_random_direction(DX, DY, Dir),
18     NX is X + DX, NY is Y + DY,
19     ( \+ claimed_pos(NX, NY)
20     -> NewCount is Count + 1,
21         NewID = NewCount,
22         assertz(room(NewID, NX, NY)),
23         assertz(room_type(NewID, NextType)),
24         assertz(connected(ParentID, NewID, Dir)),
25         assertz(claimed_pos(NX, NY)),
26         opposite_dir_val(Dir, OppDir),
27         assertz(connected(NewID, ParentID, OppDir)),
28         NewQueueIn = [room(NewID, NextType, NX, NY) | QueueIn],
29         spawn_neighbors(ParentID, X, Y, RestTypes, NewQueueIn, QueueOut,
30                         NewCount, FinalCount, Max)
31     ; spawn_neighbors(ParentID, X, Y, RestTypes, QueueIn, QueueOut,
32                       Count, FinalCount, Max)
33     ).
```

Ovaj isječak koda prikazuje kako se rekurzivno širi struktura tamnice. Ključan je predikat `spawn_neighbors` koji osigurava da se nove sobe ne preklapaju s postojećima provjerom `claimed_pos/2`, a nasumičnost smjera postiže se permutacijom liste mogućih smjerova.

3.3. Integracija i Vizualizacija

Python skripta pokreće Prolog generator i dohvaća rezultate u obliku liste soba i veza.

Isječak koda 3: Integracija Pythona i Prologa

```
1 def generate_dungeon(self, seed: int = 42, num_rooms: int = 7) -> dict:
2     """ Generira dungeon pozivom Prologa. """
3     query = f"generate_dungeon({seed}, {num_rooms}, Result)"
4     results = list(self.prolog.query(query))
5
6     if not results:
7         raise RuntimeError("Prolog nije uspio generirati dungeon")
8
9     return self._parse_dungeon_result(results[0]['Result'])
```

Kao što je prikazano u ispisu iznad, svaka soba se vizualizira kao pločica na ekranu, a hodnici (veze) se iscrtavaju između njih. Dodatno, sadržaj soba (neprijatelji, predmeti) se također generira u Prologu (generate_room_content/1) i prenosi u Python igru.

4. Kritički osvrt

Korištenje gramatika grafova pokazalo se kao robustan pristup za generiranje tamnica.

4.1. Prednosti

- **Odvajanje strukture od algoritma:** Promjenom samo `weighted_rule` predikata možemo drastično promijeniti topologiju tamnice (linearni hodnici vs. razgranati labirinti).
- **Fino podešavanje balansa:** Težine omogućuju kontrolu rijetkih soba (shop 10%) bez mijenjanja algoritma.
- **Deklarativnost:** Prolog prirodno podržava backtracking što olakšava implementaciju ograničenja.
- **Modularnost:** Generator je potpuno odvojen od vizualizacije - može se koristiti s bilo kojim game engineom.

4.2. Nedostaci

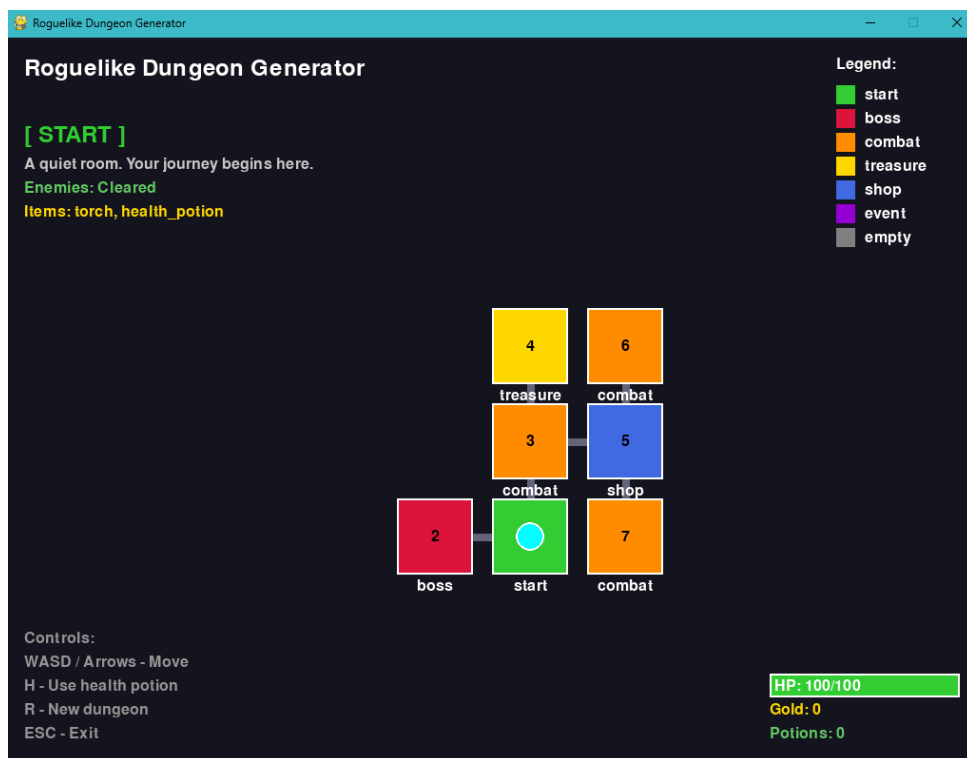
- **Složenost:** Potencijalna eksponencijalna složenost ako se pravila ne ograniče (riješeno s safety break od 500 iteracija).
- **Komunikacijski overhead:** PySwip sučelje između Pythona i Prologa uvodi određeno kašnjenje, iako za jednokratno generiranje to nije kritično.
- **Debugging:** Teže pronalaženje grešaka u Prolog kodu nego u imperativnim jezicima.
- **Ovisnost o SWI-Prologu:** Zahtijeva instalaciju Prologa na korisničkom sustavu.

4.3. Mogući nastavak

Projekt bi se mogao proširiti implementacijom lock-and-key mehanike, dinamičkim podešavanjem težine prema napretku igrača, ili vizualnim prikazom neprijatelja pomoću spriteova.

5. Prikaz rada aplikacije

U nastavku su prikazani primjeri generiranih tamnica.



Slika 2: Primjer generirane tamnice

Aplikacija omogućuje igraču kretanje kroz generirane sobe, borbu s neprijateljima i skupljanje predmeta.

6. Zaključak

U ovom radu uspješno je implementiran sustav za proceduralno generiranje razina koristeći gramatike grafova u Prologu. Sustav demonstrira moć deklarativnog programiranja u rješavanju problema generiranja strukture. Iako postoje izazovi u integraciji s imperativnim jezicima, konačni rezultat je fleksibilan i proširiv generator koji može poslužiti kao temelj za složenije igre.

Popis literature

- [1] J. Dormans, „Adventures in level design: generating missions and spaces for action adventure games,” *Proceedings of the 2010 Workshop on Procedural Content Generation in Games*, 2010., str. 1–8.
- [2] J. Togelius, G. N. Yannakakis, K. O. Stanley i C. Browne, „Search-based procedural content generation: A taxonomy and survey,” *IEEE Transactions on Computational Intelligence and AI in Games*, sv. 3, br. 3, str. 172–186, 2011.
- [3] D. Adams, „Automatic generation of dungeons for computer games,” mag. rad, University of Sheffield, 2002.
- [4] G. Smith, J. Whitehead i M. Mateas, „Tanagra: Reactive planning and constraint solving for mixed-initiative level design,” *IEEE Transactions on Computational Intelligence and AI in Games*, sv. 3, br. 3, str. 201–215, 2011.
- [5] G. Rozenberg, *Handbook of Graph Grammars and Computing by Graph Transformation*. World Scientific, 1997., sv. 1.
- [6] A. Lindenmayer, „Mathematical models for cellular interactions in development I. Filaments with one-sided inputs,” *Journal of Theoretical Biology*, sv. 18, br. 3, str. 280–299, 1968.
- [7] P. Prusinkiewicz i A. Lindenmayer, *The Algorithmic Beauty of Plants*. Springer-Verlag, 1990.
- [8] I. Bratko, *Prolog Programming for Artificial Intelligence*, 4. izdanje. Pearson Education, 2012.

Popis slika

1.	Dijagram arhitekture sustava	5
2.	Primjer generirane tamnice	10